

Unidad VI

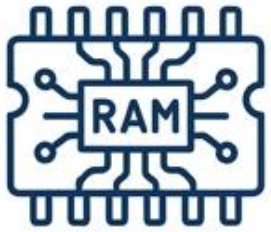
Clase 11

Árboles Avanzados

ÁRBOL B: ESTRUCTURAS PARA GRANDES VOLÚMENES

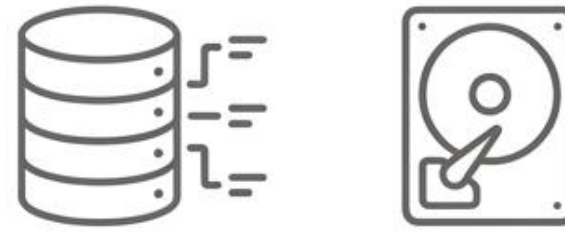
Comprender estructuras usadas en sistemas reales (bases de datos)

El problema cambia: De memoria a grandes volúmenes



Árboles en Memoria Principal

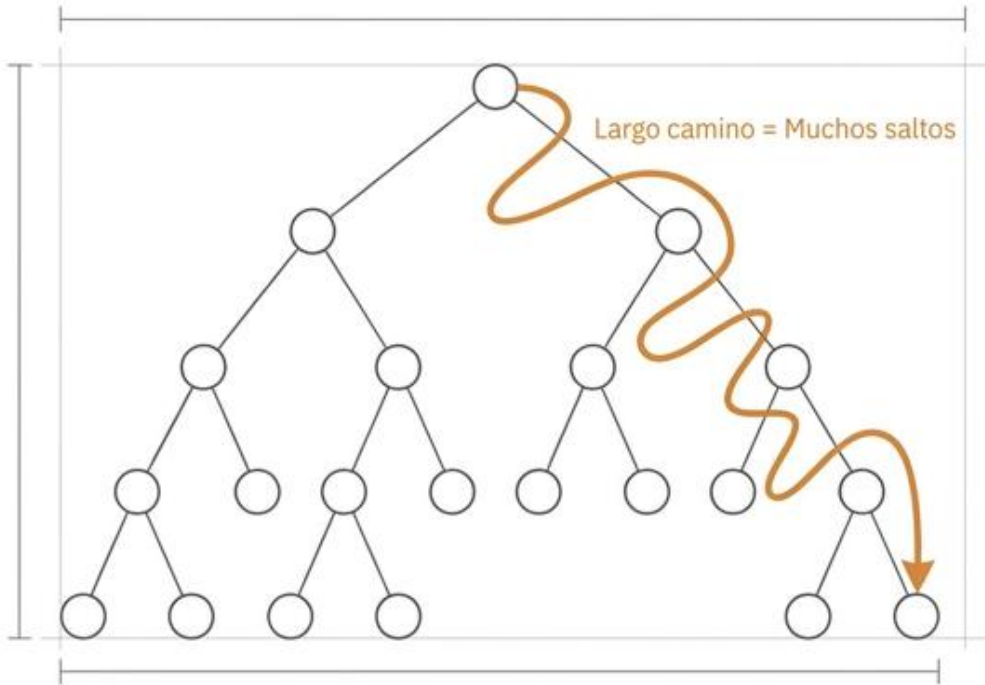
- Contexto: Estructuras tradicionales (ej. ABB, AVL).
- Velocidad: Acceso casi instantáneo.
- Foco: Minimizar la cantidad teórica de comparaciones.



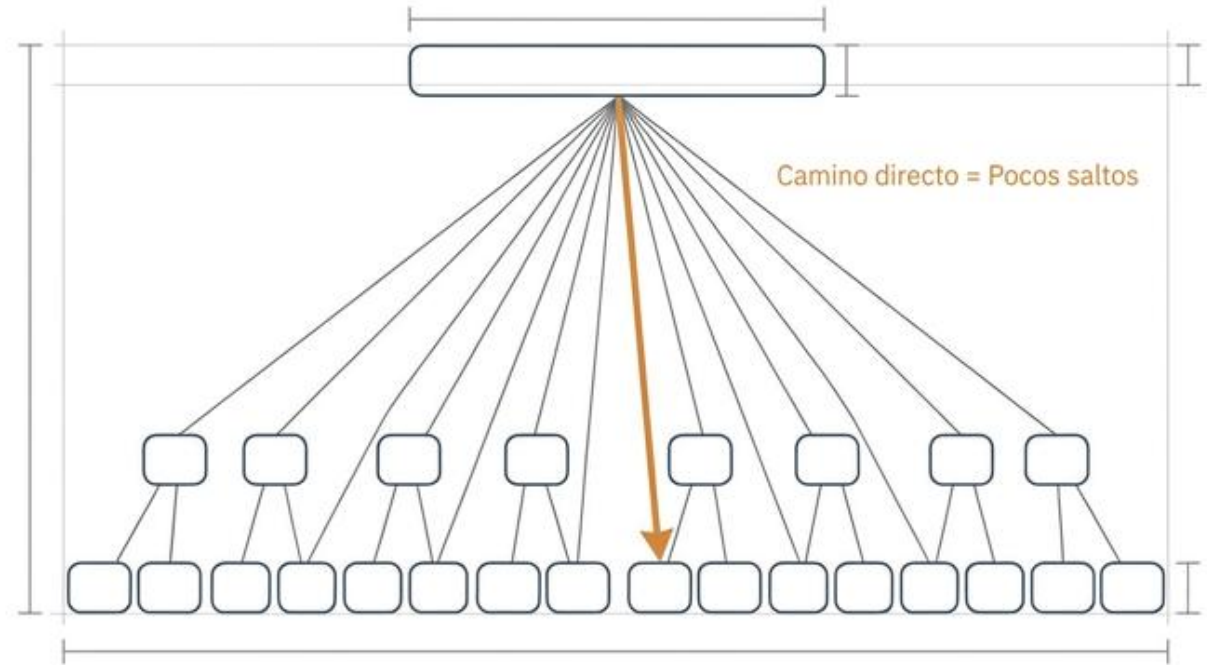
Árboles en Disco Físico (Sistemas Reales)

- Contexto: Bases de datos, índices, grandes volúmenes.
- Almacenamiento: Datos agrupados en bloques o páginas.
- Foco: Cada acceso a disco es un cuello de botella. Reducir las lecturas físicas.

El costo real de acceso a los datos

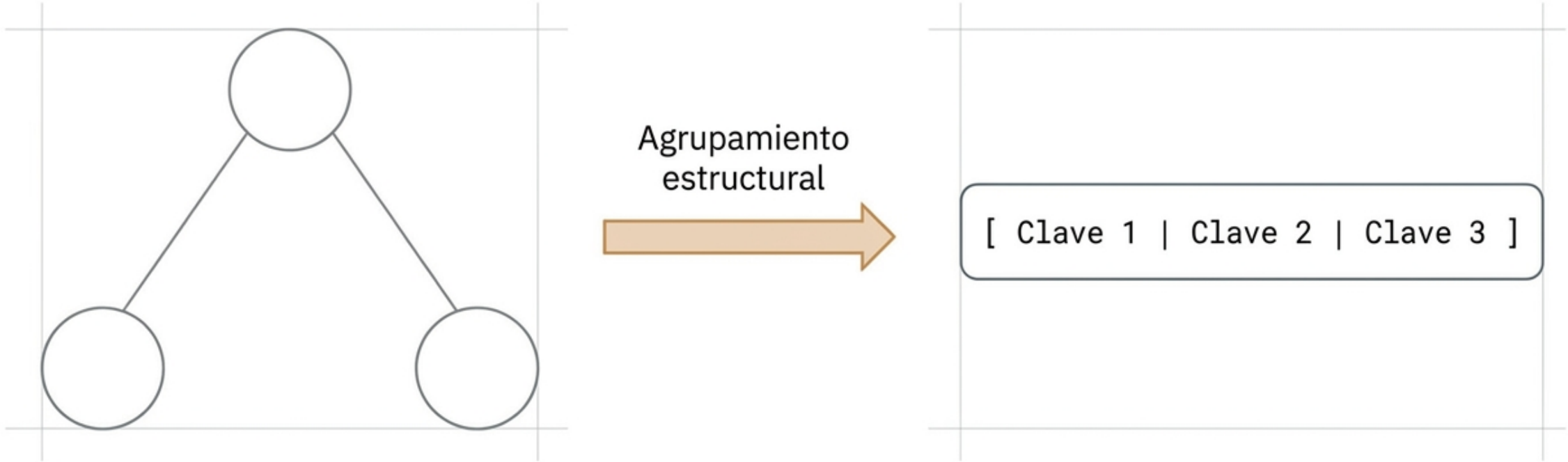


Múltiples saltos = Múltiples lecturas de disco = Lento



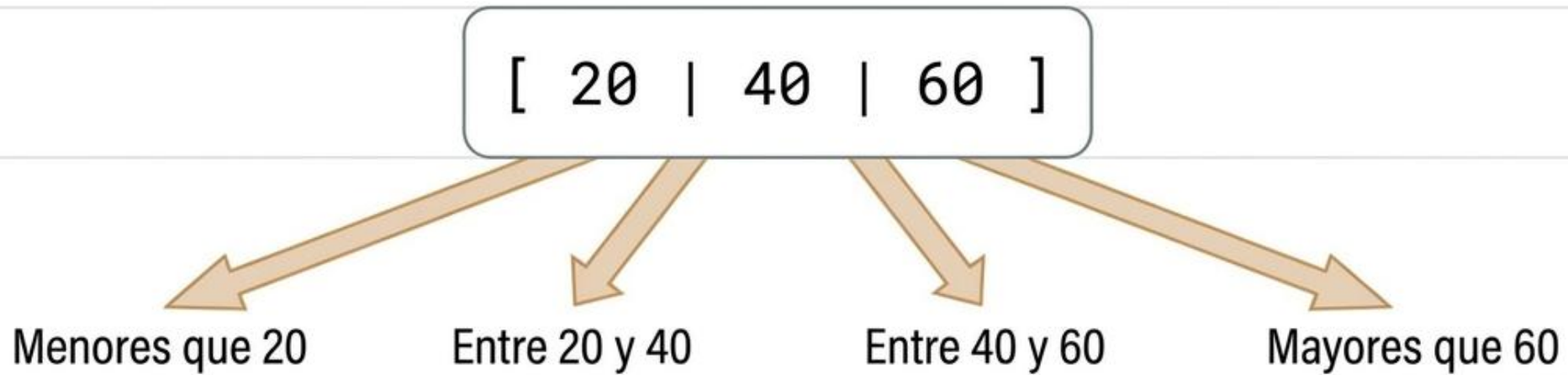
Menos saltos = Menos lecturas = Rápido

La solución: Agrupar para aplanar



- La altura del árbol es el enemigo del rendimiento en disco.
- Para reducir la altura, un nodo debe poder tomar más de dos decisiones.
- Esto se logra almacenando múltiples claves ordenadas en un solo nodo.

El nodo como página de almacenamiento



Puntos Clave

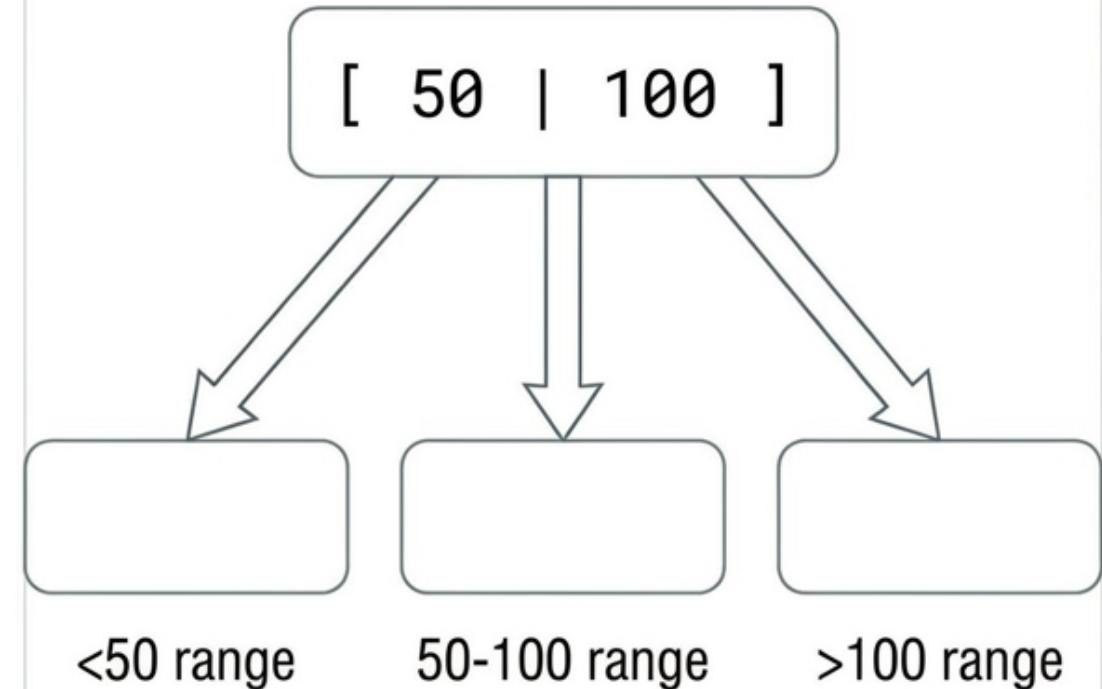
- Una página de base de datos guarda varias claves y referencias.
- Una sola lectura a disco permite comparar contra múltiples valores simultáneamente.

Aplicaciones concretas

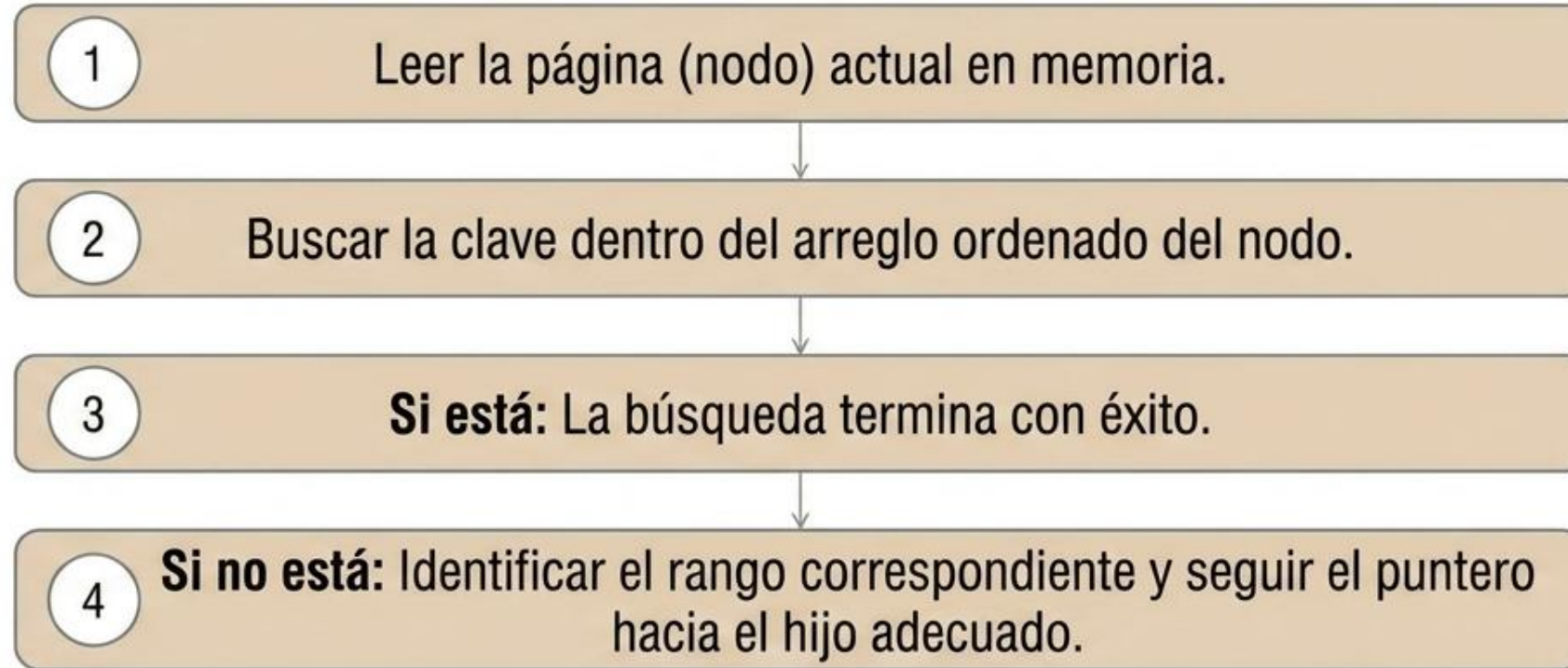
Índices de productos, facturas, clientes o envíos logísticos.

Árbol B como árbol multcamino

- Estructura de búsqueda balanceada y multcamino.
- Claves internas siempre en orden creciente.
- Los hijos representan intervalos cerrados de valores.
- La búsqueda no se limita a izquierda o derecha; se elige entre múltiples rangos posibles de una sola vez.



Navegación y Búsqueda: Toma de decisión

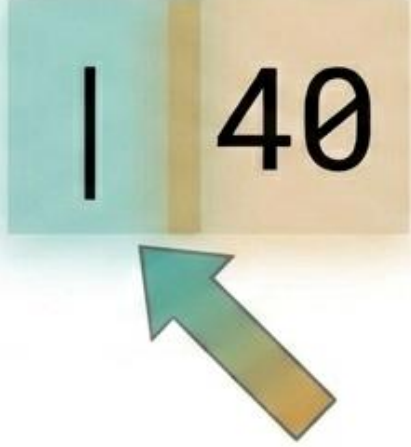


Cada nodo permite descartar varios rangos completos de una sola vez.

Ejemplo de Búsqueda: Rango Interno

Escenario: Buscar 35

[20 | 40 | 60]



- 35 es mayor que 20.
- 35 es menor que 40.
- Acción: Se continúa la búsqueda por el hijo ubicado entre 20 y 40.

Ejemplo de Búsqueda: Límite Superior

Escenario: Buscar 75

[20 | 40 | 60]



- 75 es mayor que 60 (la última clave del nodo).
- Acción: Se continúa la búsqueda directamente por el hijo derecho extremo.
- Condición de corte: La búsqueda termina al encontrar la clave, o al llegar a una hoja donde no está.

Inserción: Lógica General



Aclaración: Siempre se comienza intentando insertar en el nivel de las hojas.
El árbol crece hacia arriba, no hacia abajo.

**“La inserción puede propagarse hacia arriba,
pero el árbol conserva su balance.”**

Caso 1: Inserción sin división

Situación: El nodo hoja tiene espacio disponible.



Nota: El nodo sigue siendo válido porque no supera el límite de capacidad.
Representa una inserción rápida en una página con espacio.

“Mientras haya espacio, insertar es ordenar dentro del nodo.”

Caso 2: Inserción con división y promoción

Concepto: Cuando un nodo supera su capacidad máxima, no puede almacenar la nueva clave. Debe dividirse.

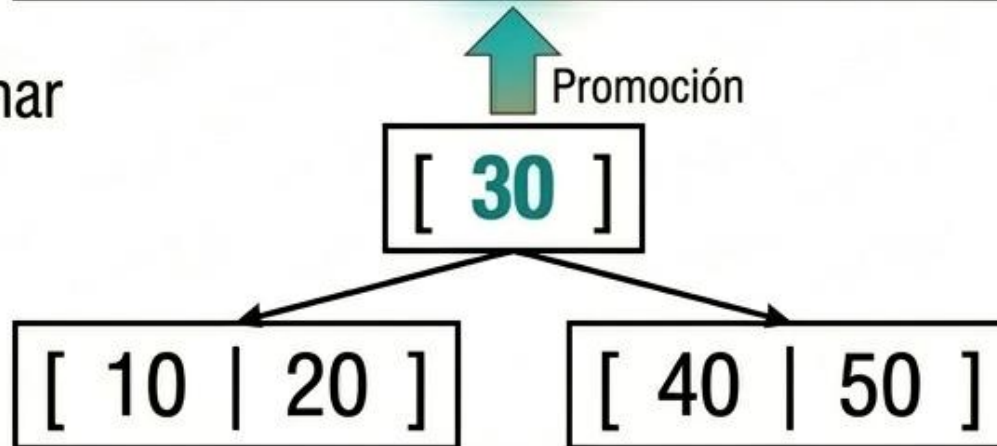
1. Exceso (Conceptual)

[10 | 20 | 30 | 40 | 50]

2. Identificar el medio

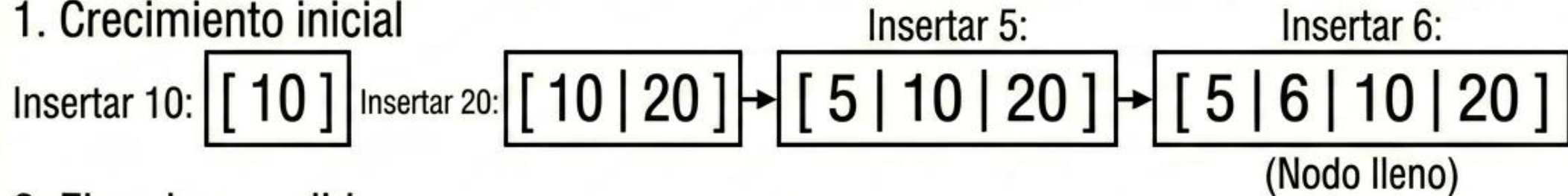
[10 | 20 | 30 | 40 | 50]

3. Dividir y Promocionar



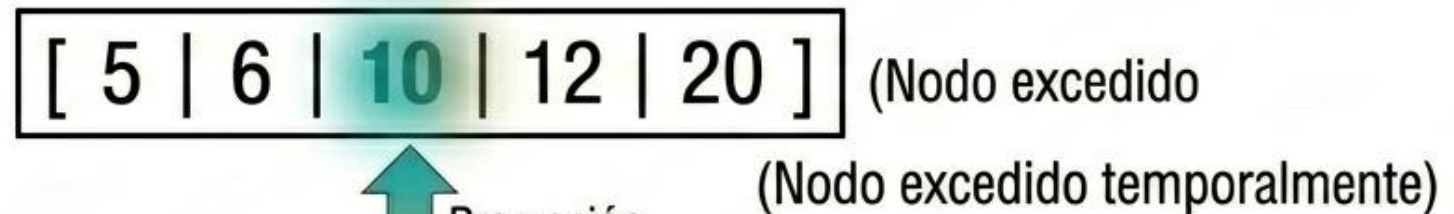
Secuencia evolutiva de crecimiento (Árbol B de Orden 5)

1. Crecimiento inicial



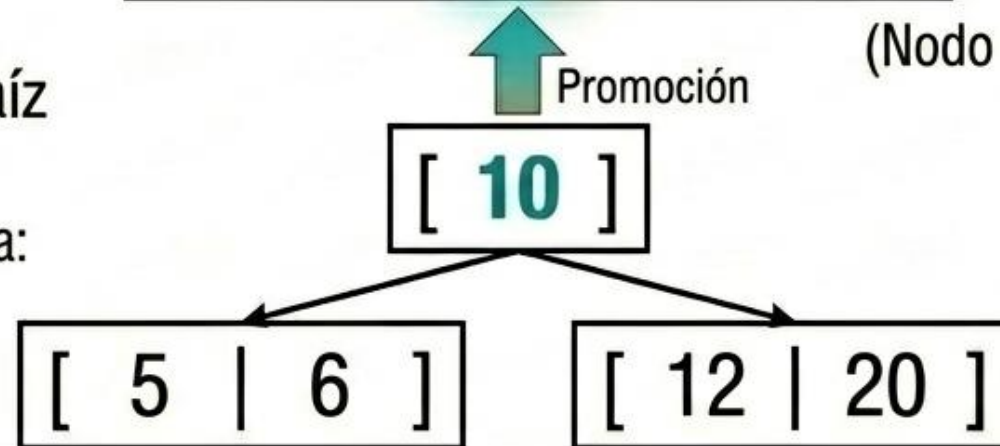
2. El nodo excedido

Insertar 12 genera
excedente temporal:



3. División y nueva raíz

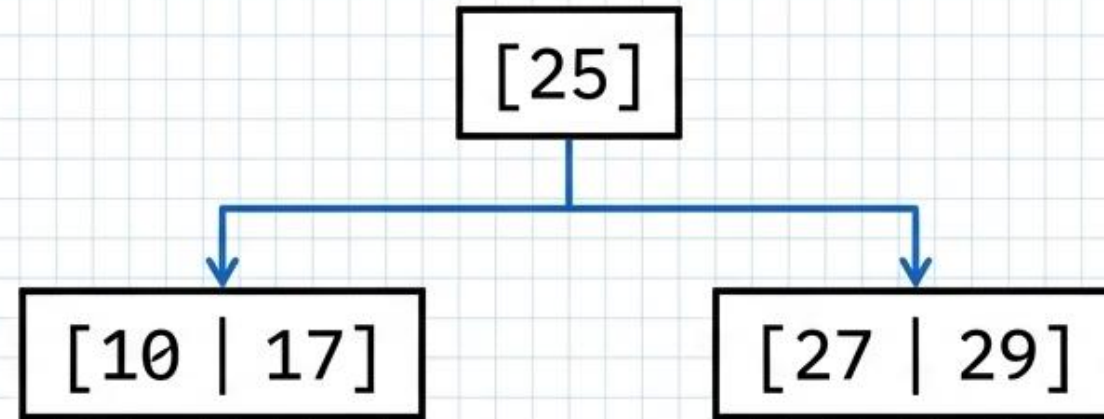
Se promociona el 10.
El árbol crece en altura:



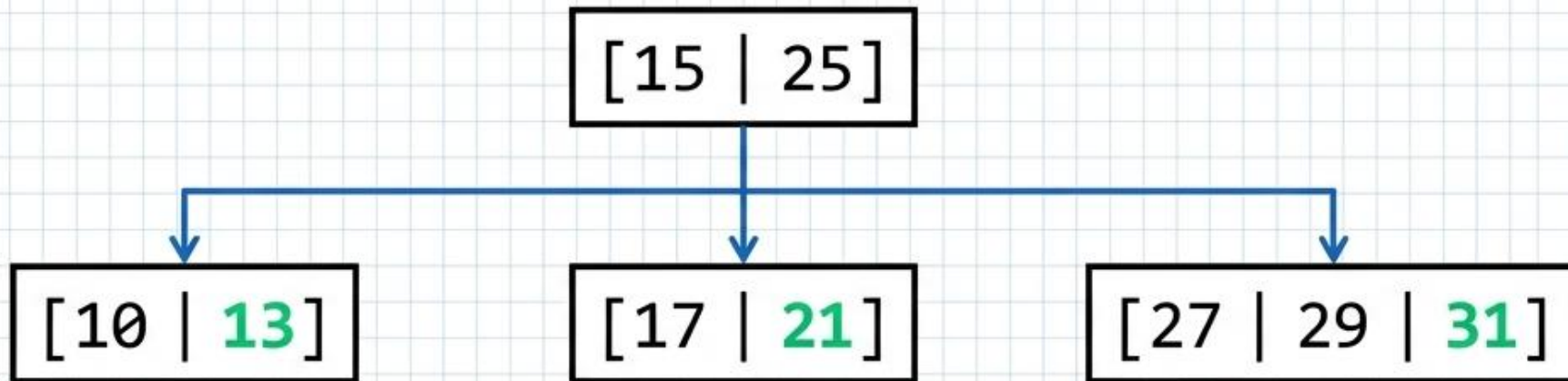
“El Árbol B aumenta su capacidad de decisión sin generar ramas largas.”

Inserciones Posteriores: Crecimiento Controlado

Estado Inicial



Evolución

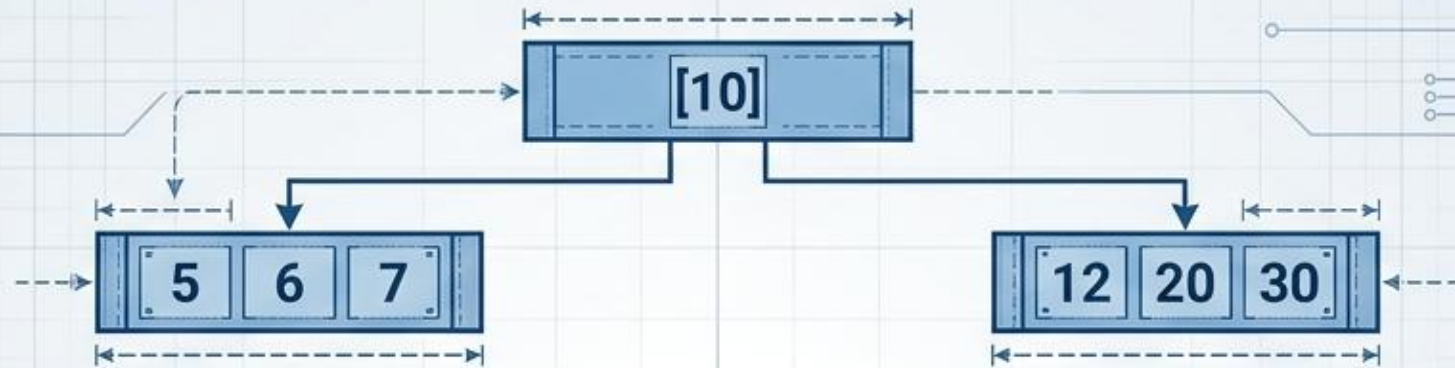


El Árbol B aumenta su capacidad de decisión sin generar ramas largas

División en Nodo Interno

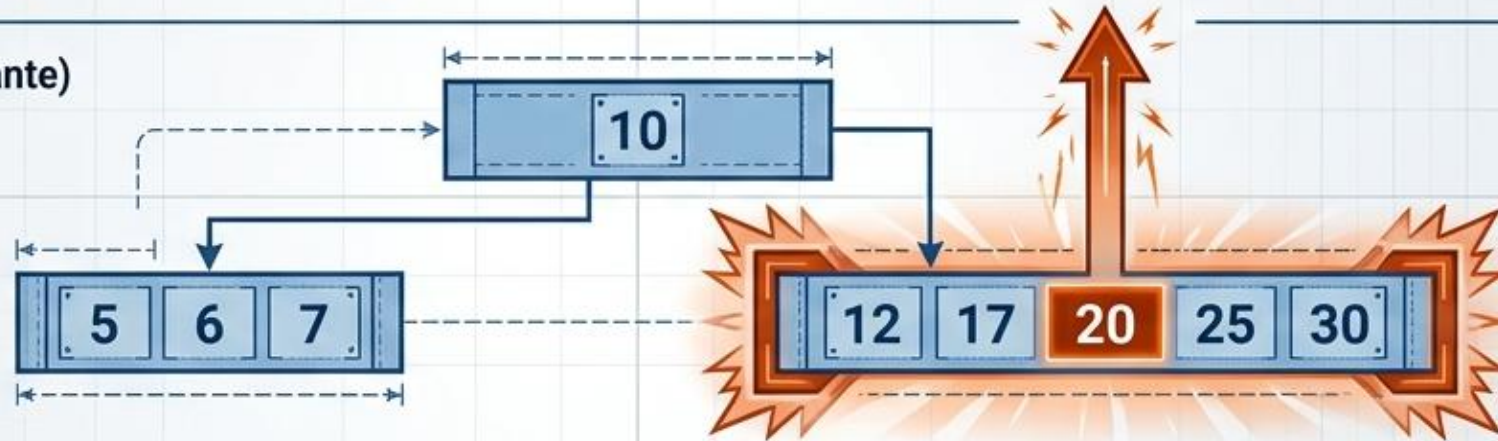
Data Architect Blueprint

Paso 1: Punto de Partida



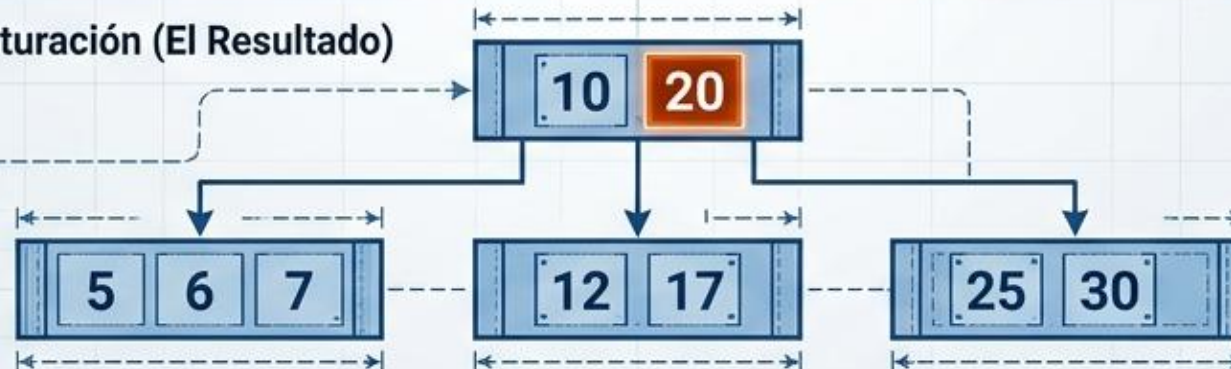
Paso 2: Sobrecarga (El Detonante)

Se insertan 17 y 25.
El hijo derecho se excede.



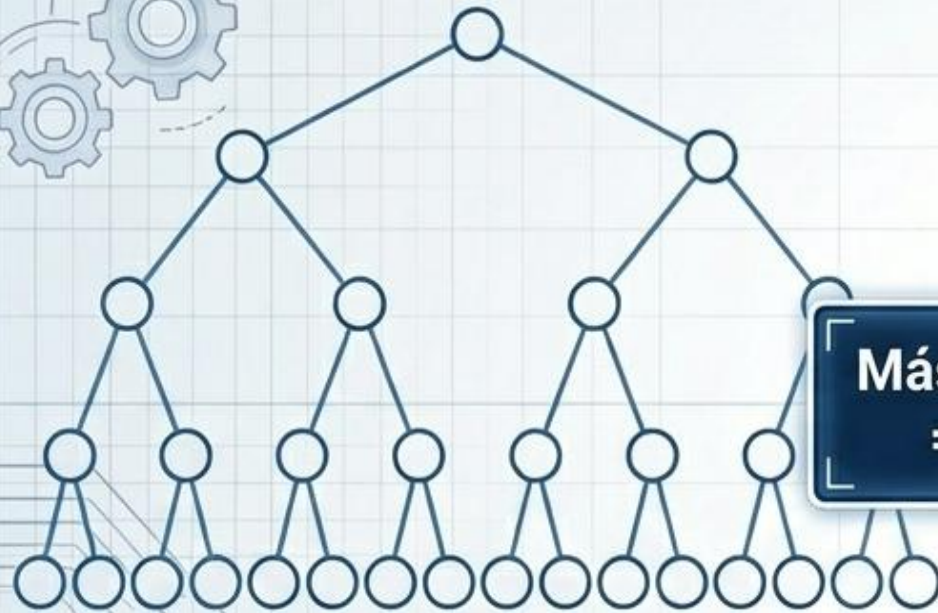
Paso 3: Promoción y Reestructuración (El Resultado)

La clave 20 sube al padre.



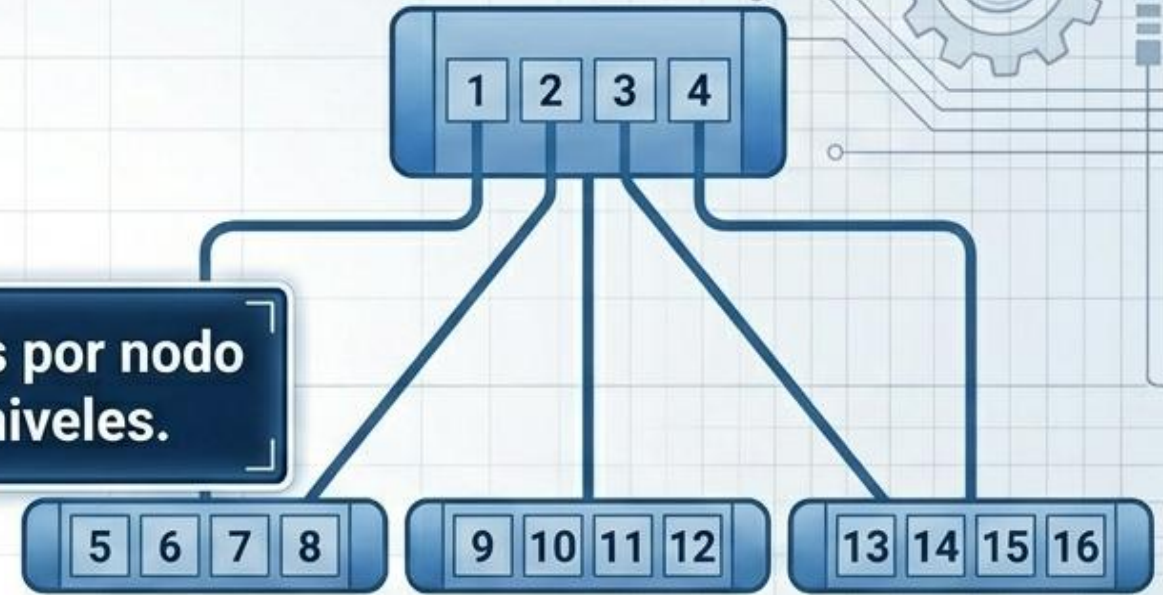
Cuando un hijo se divide,
el padre incorpora una
nueva clave separadora.

Árbol Binario (Profundo)



Más caminos por nodo
= Menos niveles.

Árbol B (Plano)



- Cada nodo = múltiples caminos.
- Divide el espacio en rangos masivos.
- Mínimos accesos a disco.

- Cada nodo = 2 caminos.
- Requiere recorrer muchos niveles.
- Múltiples accesos a disco físico.

El Árbol B aplanla la estructura cambiando **profundidad** por **amplitud**.

Eliminación Conceptual: Ocupación Mínima

Data Architect Blueprint

Regla Fundamental: Un nodo interno no puede quedar casi vacío. Debe mantener al menos $p/2$ claves.

Índice Válido



Página eficiente. Rangos de búsqueda correctamente separados.

Estado de Underflow

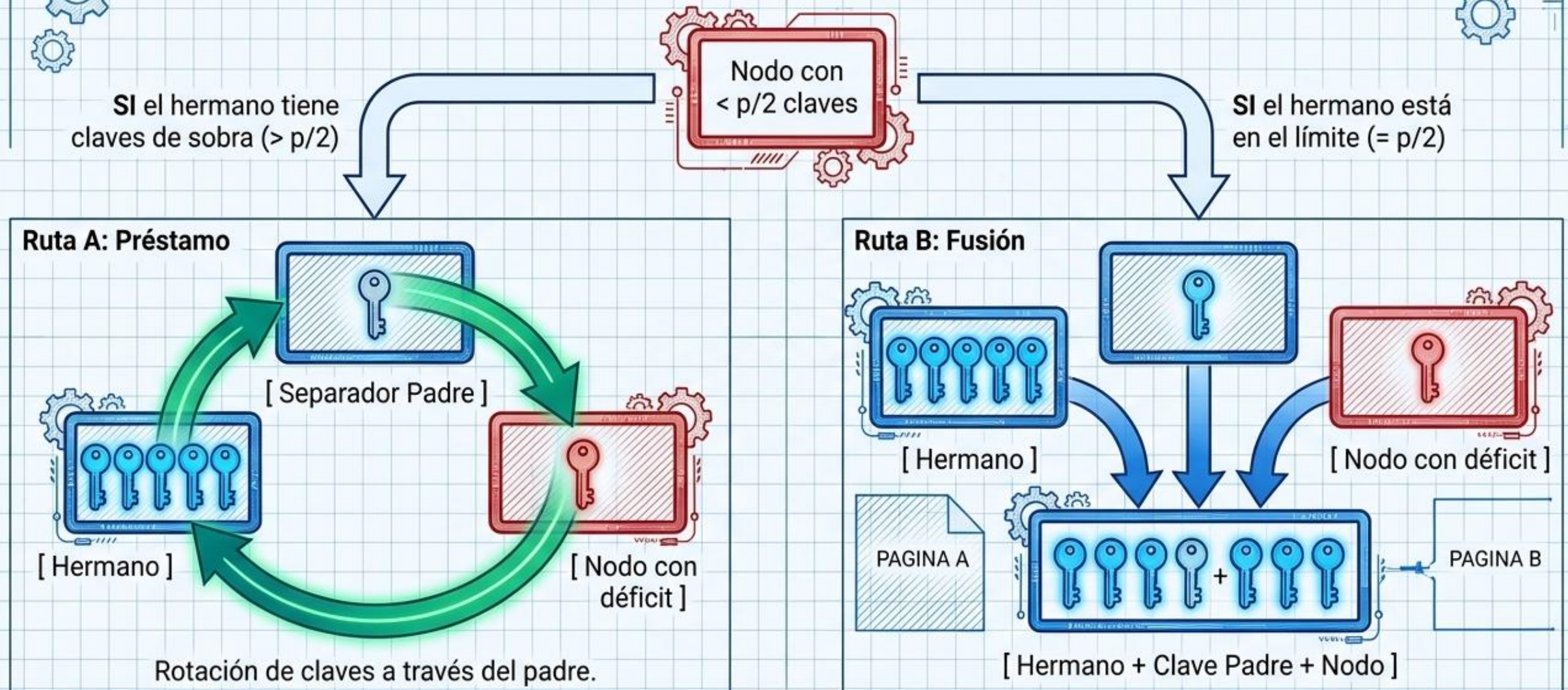


Las claves separan rangos de búsqueda. Si eliminamos demasiadas claves sin reestructurar, el árbol pierde su propósito.

Página ineficiente. Rompe la arquitectura del índice de la base de datos.

Eliminar en un Árbol B no es solo borrar: es garantizar que la página siga siendo un **índice válido**.

Resolución de Underflow: Préstamo y Fusión



Préstamo y fusión permiten recuperar la ocupación mínima sin romper el orden del árbol.

Eliminación desde Nodo Interno

El Problema:

Si se elimina una clave interna, no se puede dejar un hueco. Esa clave separa rangos enteros y guía la búsqueda.

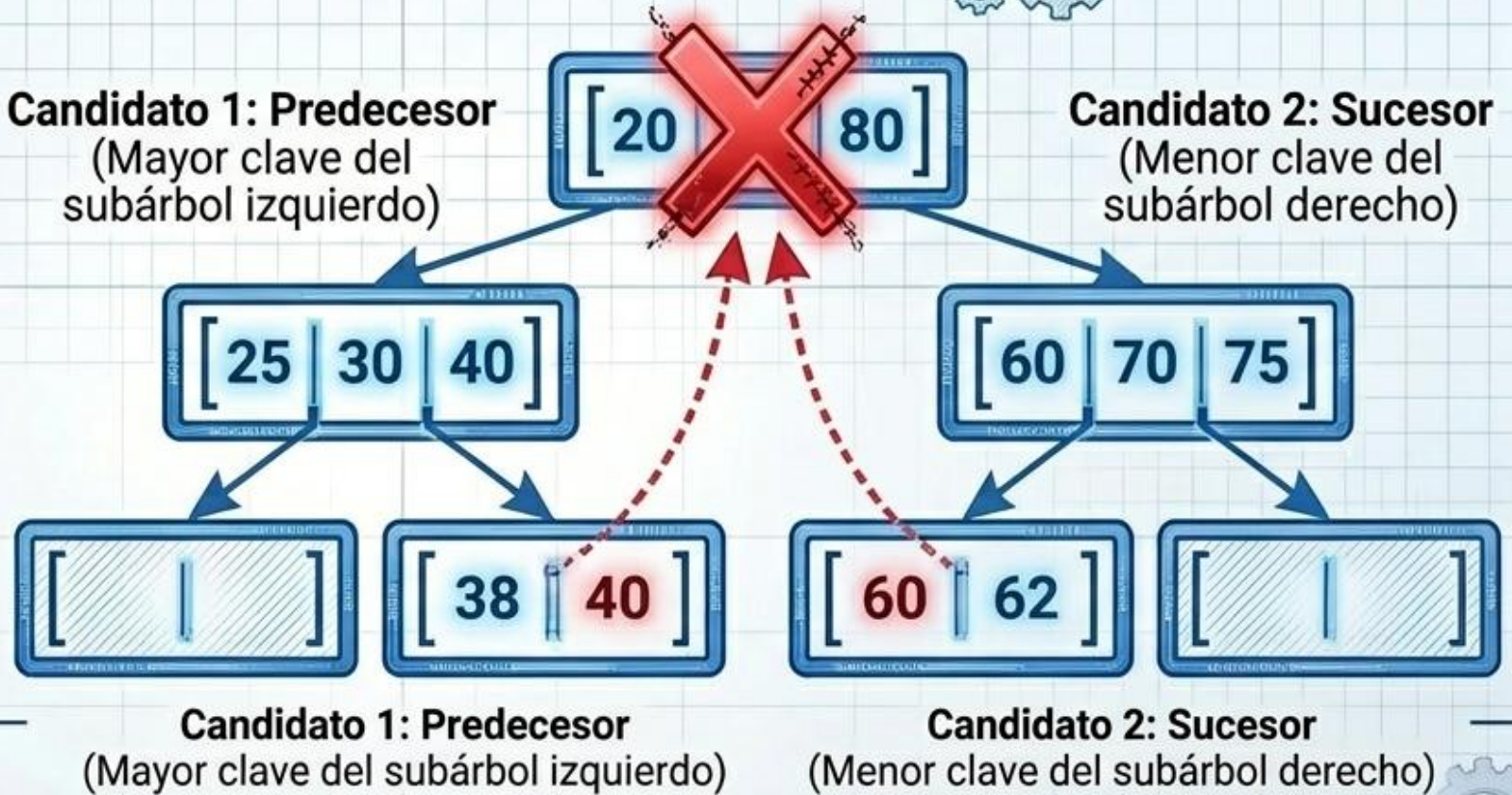
Solución:

Se reemplaza por una clave equivalente para mantener intacta la jerarquía.

Cirugía de Datos

Candidato 1: Predecesor
(Mayor clave del subárbol izquierdo)

Candidato 2: Sucesor
(Menor clave del subárbol derecho)



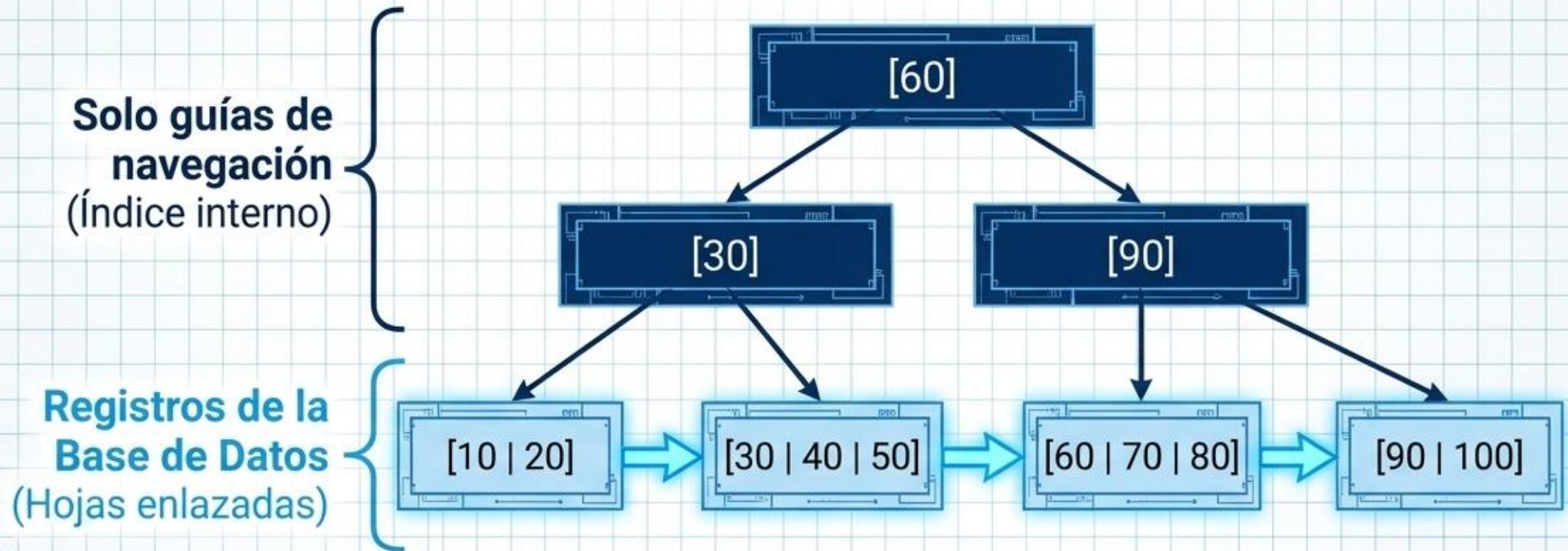
Las claves **internas** son **guías**; si se eliminan, deben reemplazarse por el valor **más próximo** de sus hojas.

Evolución y Especialización: El Árbol B+



El Árbol B+ conserva los nodos múltiples y la baja altura, pero cambia radicalmente la **ubicación de los datos reales**. Es la optimización definitiva para bases de datos.

El Plano del B+



El Árbol B+ usa los nodos internos como índice y concentra los datos en hojas.

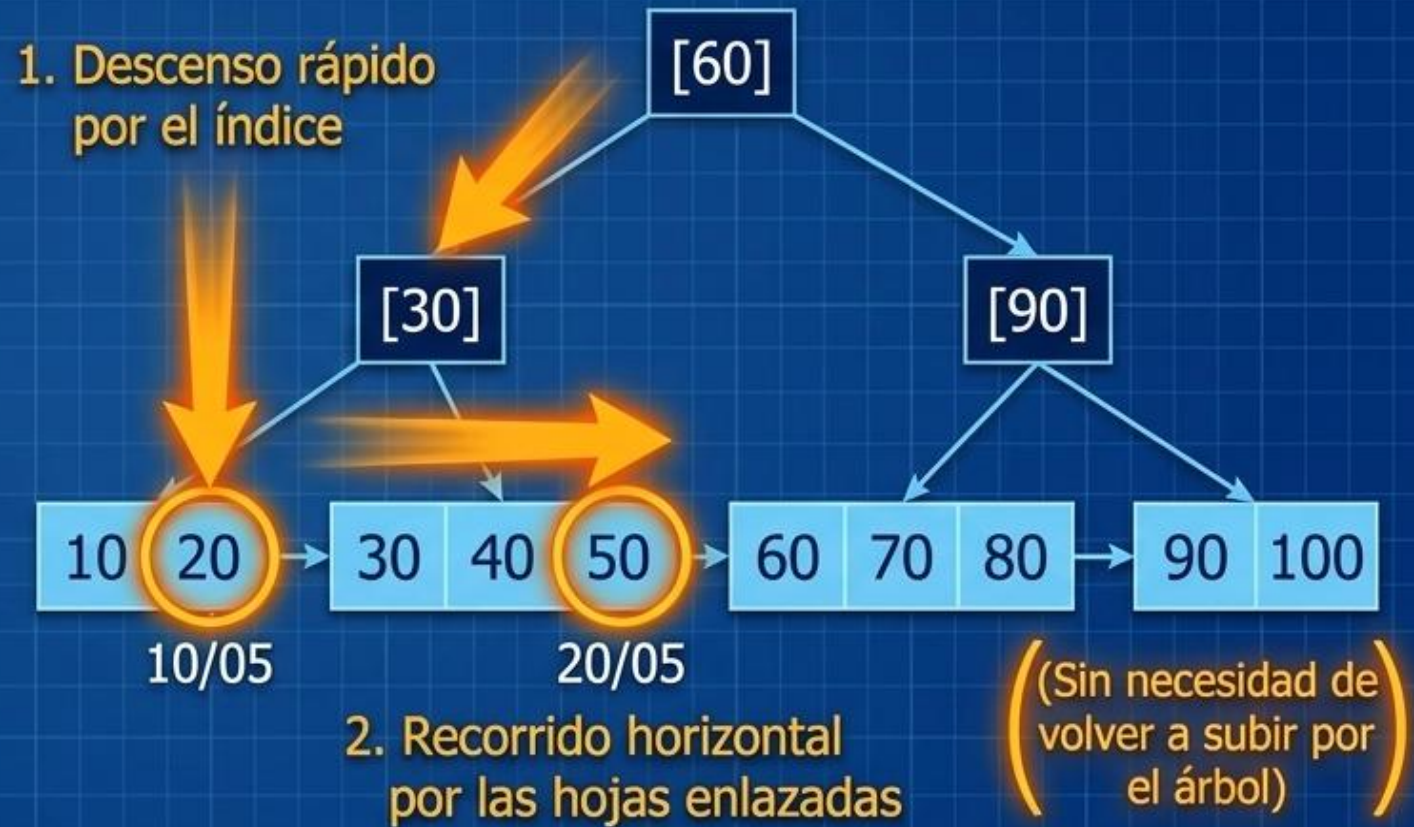
Árbol B+ y Búsquedas por Rango

Casos de Uso Reales:

Las consultas en Bases de Datos no suelen buscar un único valor.

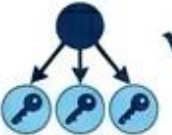
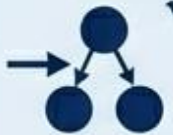
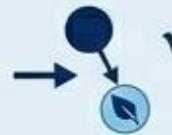
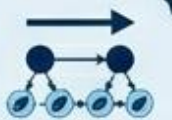
- Fechas (Ej: 10/05 al 20/05)
- Rango de importes en facturación
- Envíos programados de una semana

Mapa de Recorrido Secuencial



El B+ es especialmente eficiente cuando la consulta requiere recorrer datos ordenados.

Diagnóstico: Árbol B vs. Árbol B+

Árbol B	Árbol B+
 ✓ Ubicación de claves: Nodos internos y hojas.	 ✓ Ubicación de datos: Concentrados exclusivamente en las hojas.
 ✓ Fin de la búsqueda: Puede terminar prematuramente en un nodo interno.	 ✓ Fin de la búsqueda: Llega siempre hasta el nivel de la hoja.
 ✓ Caso Óptimo: Búsquedas puntuales eficientes.	 ✓ Estructura especial: Nodos internos son solo índices; hojas enazadas horizontalmente.
 ✓ Mecánica principal: Reduce altura agrupando claves.	 ✓ Caso Óptimo: Estándar para bases de datos; facilita recorridos por rango.

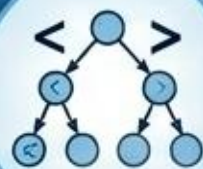
El Árbol B reduce altura; el Árbol B+ potencia además los recorridos ordenados.

Cierre Integrador: Familias de Árboles Vistas

Parada 1:
Árbol Genérico n-ario.
Jerarquías generales,
cantidad variable de hijos.



Parada 3:
ABB (Árbol Binario de Búsqueda).
Regla estricta de orden.
Búsqueda jerárquica.



Parada 5:
Árbol B. Multicamino.
Agrupar claves, reduce
accesos a disco.



Parada 2:
Árbol Binario.
Límite máximo
de dos hijos.
Base estructural.



Parada 4:
AVL.
ABB balanceado.
Altura controlada
por rotaciones.



Parada 6:
Árbol B+.
Datos en hojas
enlazadas.
Estándar para
bases de datos.



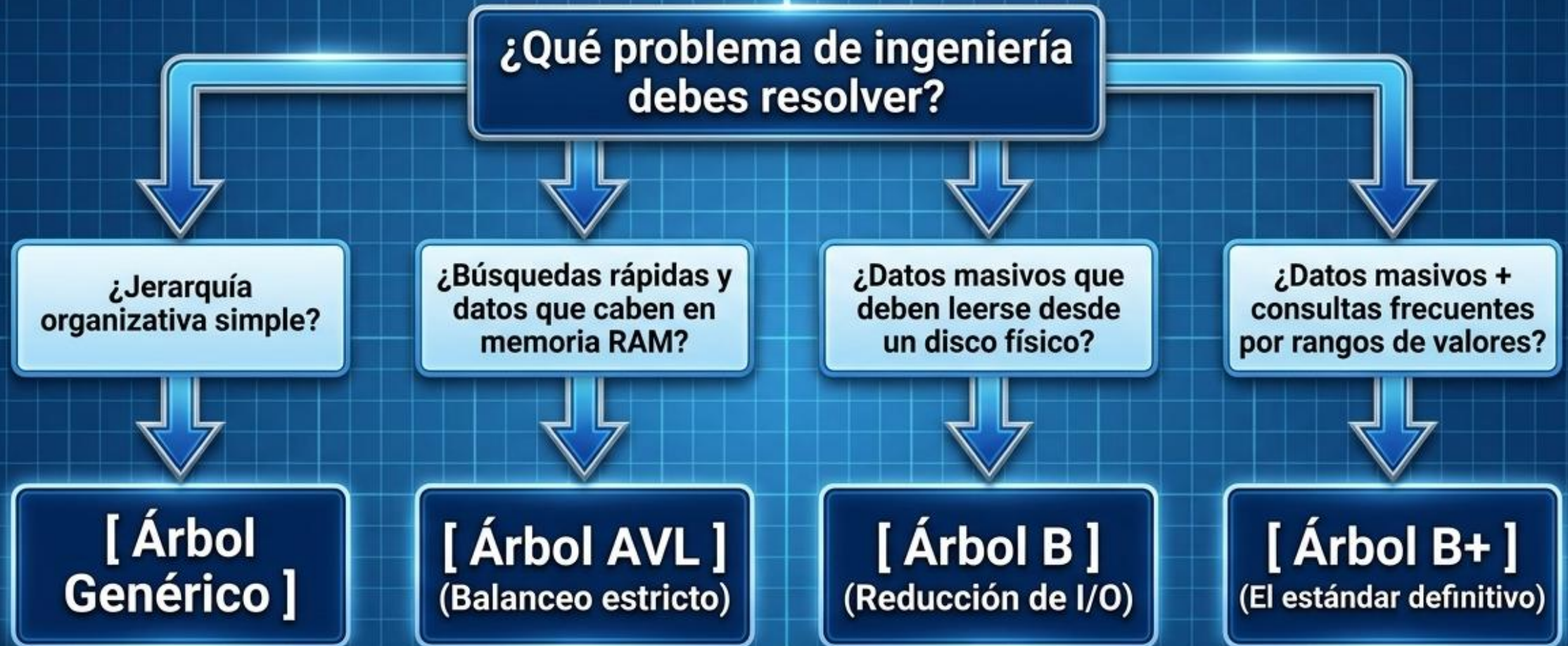
No todos los árboles resuelven el mismo problema: cada estructura responde a una necesidad distinta.

Comparativa Técnica de Árboles

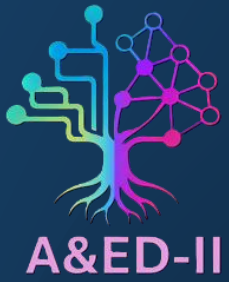
Estructura	Organización	Problema que resuelve	Ventaja principal	Limitación principal
Genérico n-ario	Hijos variables	Jerarquías generales	Flexibilidad	Sin orden de búsqueda
Binario	Máx 2 hijos	Base de recorridos	Simplicidad	No garantiza orden
ABB	Binario ordenado	Búsqueda ordenada	Recorridos eficientes	Riesgo de desbalanceo
AVL	Binario balanceado	Búsqueda rápida garantizada	Balance automático en RAM	Costo de rotaciones
Árbol B	Páginas multicamino	Grandes volúmenes en disco	Reduce accesos a disco físico	Implementación compleja
Árbol B+	Hojas enlazadas	Motores de bases de datos	Recorridos por rangos masivos	Redundancia de índices

La elección de una estructura depende del tipo de datos, el volumen, el patrón de búsqueda y el medio de almacenamiento.

Síntesis Final: Elegir la Estructura Adecuada



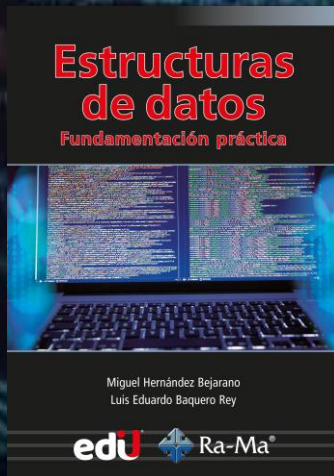
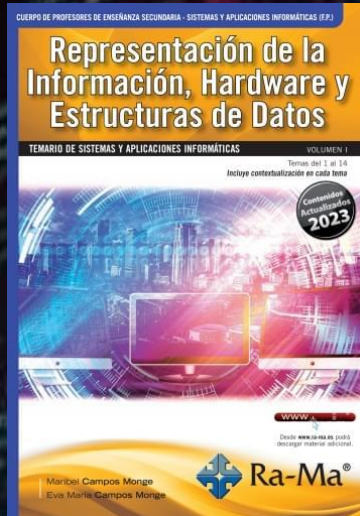
Diseñar estructuras de datos implica comprender el problema antes de elegir la solución.



Actividad

Diseñar una solución basada en TDA que utilice un Árbol B como índice para asistir la captura, almacenamiento y búsqueda de grandes volúmenes de eventos logísticos. El objetivo no es programar una base de datos completa, sino modelar cómo una estructura de datos puede mejorar el acceso a registros almacenados externamente.

Una empresa de logística regional recibe diariamente miles de eventos asociados al seguimiento de paquetes. Cada evento representa una novedad del ciclo de vida de un envío: recepción en depósito, clasificación, salida a distribución, llegada a centro logístico, intento de entrega, entrega final o devolución.



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 12

Modelar relaciones complejas: Grafos

Definición de grafo.

Matriz de adyacencia.

Lista de adyacencia.

Especificación del TDA.